

DrainFlow: Code Segment and Function Explanation Report

This report maps the MATLAB code segments of DrainFlow to their GUI role and engineering calculation function. Line ranges are based on the uploaded source file. The structure follows the calculation flow: rainfall → time of concentration → IDF intensity → Rational Method → Manning hydraulics → normal depth → design verdict.

Header, project description, and engineering overview

Approximate code line range: 1 - 70

```
1: ``
2: function DrainFlow
3: %DRAINFLOW Live storm sewer design assistant - single-file application.
4: %
5: %   DrainFlow
6: %
7: %   ONE FILE, NO DEPENDENCIES. Everything the application needs - the IDF
8: %   table, the Rational Method, Manning's equation, the circular-pipe
9: %   geometry, the bisection root finder, the design storm, the numerical
10: %   integrators and the statistics - lives in this file as local functions,
11: %   in the same way a web page carries its own CSS and JavaScript. Drop
12: %   this file anywhere on the MATLAB path and type DrainFlow.
13: %
14: %   STATION: KHULNA, BANGLADESH
15: %
16: %   -----
17: %   The rainfall data is the published Khulna IDF table:
18: %
19: %       Sheonty, S.R. & Islam, G.M.T. (2020) "Establishment of Rainfall
20: %       Intensity-Duration-Frequency Curves of Khulna". Proceedings of the
21: %       5th International Conference on Civil Engineering for Sustainable
22: %       Development (ICCESD 2020), 7-9 February 2020, KUET, Khulna,
23: %       Bangladesh. Paper WRE-4374.
24: %       https://iccesd.kuet.ac.bd/2020/Papers/WRE-4374.pdf
25: %
26: %   That study used 62 years of BMD annual maximum daily rainfall for
27: %   Khulna (1948-2010), checked the record for homogeneity, consistency
28: %   and randomness, fitted a Gumbel Type-I Extreme Value distribution, and
29: %   then disaggregated to short durations using three candidate equations
30: %   (Talbot, Sherman, Kimjima), selecting Sherman by least squares.
31: %
32: %   Durations run from 15 minutes to 24 hours, which is the range urban
33: %   storm sewer design actually needs.
34: %
35: %   CAVEAT TO DECLARE IN THE REPORT: short-duration rainfall gauges are
36: %   scarce in Bangladesh, so durations below about 3 hours are
37: %   DISAGGREGATED from daily records rather than directly measured. The
38: %   source paper states this limitation itself.
39: %
40: %   WHAT IT DOES
41: %       1   Computes the time of concentration of the catchment (Kirpich).
42: %       2   Reads a rainfall intensity from the Khulna IDF table at that
43: %           duration, interpolating between tabulated entries.
44: %       3   Converts it to a peak discharge with the Rational Method.
45: %       4   Computes the trial pipe's capacity with Manning's equation.
46: %       5   Solves Manning's equation backwards, by bisection, for the depth
47: %           the water actually reaches - and animates it.
48: %       6   Reports a pass / warn / fail verdict against capacity and the
49: %           self-cleansing velocity limits.
50: %
51: %   BUTTONS
52: %       Pause / Play      freeze or resume the animation
```

```

52: %      Print report      write a full numerical design report, including the
53: %                          hand-calculation check and the Gumbel back-analysis
54: %                          of the published table, to the Command Window
55: %
56: %      REQUIRES MATLAB R2019a or newer (uifigure with uigridlayout).
57: %      No toolboxes.
58: %
59: %      CONTENTS OF THIS FILE
60: %      DrainFlow ..... the application (this function)
61: %      nested functions ..... callbacks, solver cache, drawing
62: %      -- user interface helpers --
63: %      sectionLabel ..... heading in the control column
64: %      styleAxes ..... dark theme for a uiaxes
65: %      -- engineering functions ---
66: %      idf_intensity ..... Khulna IDF table + interp1 interpolation
67: %      time_of_concentration ..... K
... (remaining code in this segment omitted in preview)

```

Colour palette and global constants (GUI theme + hydraulic limit)

Approximate code line range: 71 - 100

```

71: %      manning_capacity ..... full-pipe capacity, Qmax by grid search
72: %      bisect_root ..... bisection root finder (from the lectures)
73: %      normal_depth ..... depth from discharge - the root problem
74: %      triangular_hyetograph ..... design storm shape
75: %      simpson13 ..... Simpson's 1/3 rule
76: %      design_status ..... pass / warn / fail verdict
77: %      gumbel_fit ..... EV-I fit by the method of moments
78: %      gumbel_from_table ..... recover EV-I parameters from the table
79: %
80: %      CE 206 - DrainFlow: Storm Sewer Design Assistant
81: %      Bangladesh University of Engineering and Technology
82: %
83: %% =====
84: % COLOUR PALETTE
85: % =====
86: COL.bg      = [0.039 0.059 0.110]; % page background
87: COL.panel   = [0.067 0.102 0.173]; % panel fill
88: COL.panel2  = [0.051 0.082 0.141]; % inset / axes fill
89: COL.line    = [0.180 0.220 0.290]; % borders and grid
90: COL.ink     = [0.910 0.929 0.969]; % primary text
91: COL.inkDim  = [0.549 0.604 0.722]; % secondary text
92: COL.water   = [0.310 0.639 0.890]; % water blue
93: COL.ok      = [0.208 0.761 0.584]; % green
94: COL.warn    = [0.910 0.639 0.239]; % amber
95: COL.danger  = [0.886 0.333 0.310]; % red
96: %
97: MONO = get(groot, 'FixedWidthFontName');
98: %
99: % Depth at which discharge in a circular pipe is a maximum, as a fraction
100: % of the diameter. Q(y) rises then FALLS: past this point the wall curves

```

State variables and default engineering inputs

Approximate code line range: 101 - 135

```

101: % inward and adds wetted perimeter faster than flow area, so R = A/P drops
102: % and Q with it. The ratio is a property of the circle alone - D, S and n
103: % all cancel. Used to bracket the bisection.
104: YMAX_RATIO = 0.9381812;
105: %
106: %% =====
107: % STATE
108: % =====
109: st.A      = 2.0; % catchment area (ha)
110: st.C      = 0.65; % runoff coefficient

```

```

111: st.L      = 600;      % longest overland flow path (m)
112: st.Sc     = 0.005;   % average catchment slope (m/m)
113: st.useTc  = true;    % set the storm duration equal to tc
114: st.Tr     = 10;      % return period (yr)
115: st.durMin = 30;      % storm duration (MINUTES) when useTc is false
116: st.override = false; % use a manual intensity instead of the IDF table
117: st.manualI = 120;    % manual intensity (mm/hr)
118: st.D      = 0.75;    % pipe diameter (m)
119: st.S      = 0.005;   % pipe slope (m/m)
120: st.n      = 0.013;   % Manning's n
121: st.loopSec = 10;     % real seconds per simulated storm
122: st.vmin    = 0.6;     % self-cleansing velocity limit (m/s)
123: st.vmax    = 3.0;     % scour velocity limit (m/s)
124:
125: st.simT     = 0;      % simulation clock, 0 to 1
126: st.running  = true;
127: st.hist     = struct('t', [], 'i', [], 'q', []);
128: st.rain     = struct('x', [], 'y', [], 'len', [], 'v', []);
129:
130: % Derived, filled in by recompute()
131: st.tc = 0; st.durUse = 30; st.exact = false;
132: st.Ipeak = 0; st.Qreq = 0; st.Qfull = 0; st.Vfull = 0;
133: st.Qmax = 0; st.yQmax = 0; st.yflow = 0; st.Vflow = 0;
134: st.status = ''; st.level = 1;
135:

```

GUI window, grid layout, and title section

Approximate code line range: 136 - 190

```

136: %% =====
137: % WINDOW AND TOP-LEVEL GRID
138: % =====
139: fig = uifigure('Name', 'DrainFlow - Storm Sewer Design Assistant (Khulna)', ...
140:               'Color', COL.bg, 'Position', [50 50 1360 850]);
141:
142: mainGrid = uigridlayout(fig, [2 2]);
143: mainGrid.RowHeight = {52, '1x'};
144: mainGrid.ColumnWidth = {340, '1x'};
145: mainGrid.BackgroundColor = COL.bg;
146: mainGrid.Padding = [18 14 18 10];
147: mainGrid.RowSpacing = 12;
148: mainGrid.ColumnSpacing = 16;
149:
150: hdr = uigridlayout(mainGrid, [1 2]);
151: hdr.Layout.Row = 1;
152: hdr.Layout.Column = [1 2];
153: hdr.ColumnWidth = {'1x', 360};
154: hdr.Padding = [0 0 0 0];
155: hdr.BackgroundColor = COL.bg;
156:
157: titleLbl = uilabel(hdr, 'Text', 'DrainFlow', 'FontSize', 24, ...
158:                   'FontWeight', 'bold', 'FontColor', COL.link);
159: titleLbl.Layout.Column = 1;
160:
161: h.badge = uilabel(hdr, 'Text', 'KHULNA IDF - SIMULATING', 'FontName', MONO, ...
162:                  'FontSize', 12, 'FontColor', COL.ok, 'HorizontalAlignment', 'right');
163: h.badge.Layout.Column = 2;
164:
165: %% =====
166: % LEFT CONTROL COLUMN
167: % =====
168: ctrlPanel = uipanel(mainGrid, 'BackgroundColor', COL.panel, ...
169:                    'BorderType', 'none');
170: ctrlPanel.Layout.Row = 2;
171: ctrlPanel.Layout.Column = 1;
172:
173: cg = uigridlayout(ctrlPanel, [27 1]);
174: cg.RowHeight = {20, 18, 32, 18, 32, 18, 32, 18, 32, 22, 22, 52, 22, 28, ...

```

```

175:                26, 18, 32, 18, 32, 16, 28, ...
176:                26, 18, 32, 30, 30, '1x'};
177: cg.Padding      = [16 12 16 12];
178: cg.RowSpacing   = 4;
179: cg.BackgroundColor = COL.panel;
180: cg.Scrollable   = 'on';
181:
182: % ---- catchment -----
183: sectionLabel(cg, 'CATCHMENT & RAINFALL', COL, 1);
184:
185: h.lblArea = uilabel(cg, 'Text', '', 'FontSize', 12, 'FontColor', COL.inkDim);
186: h.lblArea.Layout.Row = 2;
187: h.sldArea = uislider(cg, 'Limits', [0.1 20], 'Value', st.A, ...
188:     'MajorTicks', [], 'MinorTicks', [], 'FontColor', COL.inkDim);
189: h.sldArea.Layout.Row = 3;
190: h.sldArea.ValueChangingFcn = @(s,e) setField('A', e.Value);

```

Catchment & rainfall GUI controls

Approximate code line range: 191 - 320

```

191: h.sldArea.ValueChangedFcn = @(s,e) setField('A', s.Value);
192:
193: h.lblC = uilabel(cg, 'Text', '', 'FontSize', 12, 'FontColor', COL.inkDim);
194: h.lblC.Layout.Row = 4;
195: h.sldC = uislider(cg, 'Limits', [0.10 1.00], 'Value', st.C, ...
196:     'MajorTicks', [], 'MinorTicks', [], 'FontColor', COL.inkDim);
197: h.sldC.Layout.Row = 5;
198: h.sldC.ValueChangingFcn = @(s,e) setField('C', e.Value);
199: h.sldC.ValueChangedFcn = @(s,e) setField('C', s.Value);
200:
201: h.lblL = uilabel(cg, 'Text', '', 'FontSize', 12, 'FontColor', COL.inkDim);
202: h.lblL.Layout.Row = 6;
203: h.sldL = uislider(cg, 'Limits', [50 2000], 'Value', st.L, ...
204:     'MajorTicks', [], 'MinorTicks', [], 'FontColor', COL.inkDim);
205: h.sldL.Layout.Row = 7;
206: h.sldL.ValueChangingFcn = @(s,e) setField('L', e.Value);
207: h.sldL.ValueChangedFcn = @(s,e) setField('L', s.Value);
208:
209: h.lblSc = uilabel(cg, 'Text', '', 'FontSize', 12, 'FontColor', COL.inkDim);
210: h.lblSc.Layout.Row = 8;
211: h.sldSc = uislider(cg, 'Limits', [0.0005 0.0500], 'Value', st.Sc, ...
212:     'MajorTicks', [], 'MinorTicks', [], 'FontColor', COL.inkDim);
213: h.sldSc.Layout.Row = 9;
214: h.sldSc.ValueChangingFcn = @(s,e) setField('Sc', e.Value);
215: h.sldSc.ValueChangedFcn = @(s,e) setField('Sc', s.Value);
216:
217: h.lblTc = uilabel(cg, 'Text', '', 'FontSize', 12, 'FontName', MONO, ...
218:     'FontColor', COL.water);
219: h.lblTc.Layout.Row = 10;
220:
221: h.chkTc = ucheckbox(cg, 'Text', ' storm duration = t_c (recommended)', ...
222:     'Value', st.useTc, 'FontSize', 11.5, 'FontColor', COL.inkDim);
223: h.chkTc.Layout.Row = 11;
224: h.chkTc.ValueChangedFcn = @(s,e) setField('useTc', s.Value);
225:
226: % return period and duration
227: sub = uigridlayout(cg, [2 2]);
228: sub.Layout.Row = 12;
229: sub.RowHeight = {16, 28};
230: sub.ColumnWidth = {'1x', '1x'};
231: sub.Padding = [0 0 0 0];
232: sub.RowSpacing = 2;
233: sub.ColumnSpacing = 8;
234: sub.BackgroundColor = COL.panel;
235:
236: l1 = uilabel(sub, 'Text', 'Return period (yr)', 'FontSize', 11, ...
237:     'FontColor', COL.inkDim); l1.Layout.Row = 1; l1.Layout.Column = 1;
238: l2 = uilabel(sub, 'Text', 'Duration', 'FontSize', 11, ...

```

```

239:     'FontColor', COL.inkDim); l2.Layout.Row = 1; l2.Layout.Column = 2;
240:
241: h.ddTr = uidropdown(sub, 'Items', {'2','5','10','20','30','50','100'}, ...
242:     'ItemsData', [2 5 10 20 30 50 100], 'Value', st.Tr, ...
243:     'FontName', MONO, 'BackgroundColor', COL.panel2, 'FontColor', COL.ink);
244: h.ddTr.Layout.Row = 2; h.ddTr.Layout.Column = 1;
245: h.ddTr.ValueChangedFcn = @(s,e) setField('Tr', s.Value);
246:
247: h.ddDur = uidropdown(sub, ...
248:     'Items', {'15 min','30 min','60 min','90 min','2 hr','3 hr', ...
249:         '5 hr','6 hr','12 hr','24 hr'}, ...
250:     'ItemsData', [15 30 60 90 120 180 300 360 720 1440], ...
251:     'Value', st.durMin, 'Enable', 'off', ...
252:     'FontName', MONO, 'BackgroundColor', COL.panel2, 'FontColor', COL.ink);
253: h.ddDur.Layout.Row = 2; h.ddDur.Layout.Column = 2;
254: h.ddDur.ValueChangedFcn = @(s,e) setField('durMin', s.Value);
255:
256: h.chkOverride = ucheckbox(cg, 'Text', ' override intensity (mm/hr)', ...
257:     'Value', st.override, 'FontSize', 11.5, 'FontColor', COL.inkDim);
258: h.chkOverride.Layout.Row = 13;
259: h.chkOverride.ValueChangedFcn = @(s,e) setField('override', s.Value);
260:
261: h.efManual = ueditfield(cg, 'numeric', 'Value', st.manualI, ...
262:
... (remaining code in this segment omitted in preview)

```

Pipe design and playback controls

Approximate code line range: 321 - 430

```

321:     'KUET. 62 yr BMD record'; ...
322:     '(1948-2010), Gumbel EV-I with'; ...
323:     'Sherman disaggregation.'; ...
324:     'Durations under ~3 hr are'; ...
325:     'disaggregated, not gauged.'; ...
326:     ''; ...
327:     'Rational Method + Manning.'; ...
328:     'Depth solved by bisection.'; ...
329:     'Velocity limits 0.6 - 3.0 m/s.'}, ...
330:     'FontSize', 10.5, 'FontColor', COL.inkDim, ...
331:     'VerticalAlignment', 'top');
332: foot.Layout.Row = 27;
333:
334: %% =====
335: % RIGHT COLUMN - HERO PANEL AND CHARTS
336: % =====
337: rightGrid = uigridlayout(mainGrid, [2 1]);
338: rightGrid.Layout.Row = 2;
339: rightGrid.Layout.Column = 2;
340: rightGrid.RowHeight = {'1.55x', '1x'};
341: rightGrid.Padding = [0 0 0 0];
342: rightGrid.RowSpacing = 14;
343: rightGrid.BackgroundColor = COL.bg;
344:
345: heroPanel = uipanel(rightGrid, 'BackgroundColor', COL.panel2, ...
346:     'BorderType', 'none');
347: hgl = uigridlayout(heroPanel, [1 1]);
348: hgl.Padding = [0 0 0 0];
349: hgl.BackgroundColor = COL.panel2;
350:
351: h.axHero = uiaxes(hgl);
352: styleAxes(h.axHero, COL);
353: h.axHero.XLim = [0 1];
354: h.axHero.YLim = [0 1];
355: h.axHero.XTick = []; h.axHero.YTick = [];
356: grid(h.axHero, 'off');
357: h.axHero.XColor = COL.panel2;
358: h.axHero.YColor = COL.panel2;
359: hold(h.axHero, 'on');

```

```

360:
361: h.sky = patch(h.axHero, 'XData', [0 1 1 0], 'YData', [0.55 0.55 1 1], ...
362:   'FaceColor', COL.water, 'FaceAlpha', 0.05, 'EdgeColor', 'none');
363: h.rain = line(h.axHero, NaN, NaN, 'Color', [0.59 0.76 0.94], 'LineWidth', 1.1);
364: h.funnel = patch(h.axHero, 'XData', [0.13 0.39 0.28 0.24], ...
365:   'YData', [0.72 0.72 0.56 0.56], 'FaceColor', COL.ink, ...
366:   'FaceAlpha', 0.04, 'EdgeColor', COL.inkDim, 'LineWidth', 1.2);
367: h.arrow = line(h.axHero, NaN, NaN, 'Color', COL.water, ...
368:   'LineWidth', 1.6, 'LineStyle', ':');
369: h.water = patch(h.axHero, 'XData', NaN, 'YData', NaN, ...
370:   'FaceColor', COL.water, 'FaceAlpha', 0.85, 'EdgeColor', 'none');
371: h.waterLine = line(h.axHero, NaN, NaN, 'Color', [1 1 1], 'LineWidth', 1.4);
372: h.pipe = line(h.axHero, NaN, NaN, 'Color', [0.82 0.87 0.96], 'LineWidth', 2.5);
373: h.crown = line(h.axHero, NaN, NaN, 'Color', COL.inkDim, ...
374:   'LineWidth', 0.8, 'LineStyle', '--');
375:
376: h.txtFunnel = text(h.axHero, 0.13, 0.76, 'catchment', 'Color', COL.inkDim, ...
377:   'FontName', MONO, 'FontSize', 10);
378: h.txtQreqK = text(h.axHero, 0.03, 0.96, 'Required Q - Rational Method', ...
379:   'Color', COL.inkDim, 'FontName', MONO, 'FontSize', 10);
380: h.txtQreqV = text(h.axHero, 0.03, 0.905, '-', 'Color', COL.ink, ...
381:   'FontName', MONO, 'FontSize', 17, 'FontWeight', 'bold');
382: h.txtQcapK = text(h.axHero, 0.97, 0.96, 'Pipe capacity - Manning', ...
383:   'Color', COL.inkDim, 'FontName', MONO, 'FontSize', 10, ...
384:   'HorizontalAlignment', 'right');
385: h.txtQcapV = text(h.axHero, 0.97, 0.905, '-', 'Color', COL.ink, ...
386:   'FontName', MONO, 'FontSize', 17, 'FontWeight', 'bold', ...
387:   'HorizontalAlignment', 'right');
388:
389: % Rainfall intensity readout. The IDF table is the whole hydrological
390: % basis of the tool, so the value it produces is shown explicitly rather
391: % than being buried inside Q.
392: h.txtIdfV = text(h.
... (remaining code in this segment omitted in preview)

```

Hero panel and visualization objects

Approximate code line range: 431 - 650

```

431: h.capLine = line(h.axHyd, NaN, NaN, 'Color', COL.danger, ...
432:   'LineWidth', 1.5, 'LineStyle', '--');
433: h.nowLine = line(h.axHyd, NaN, NaN, 'Color', [0.75 0.79 0.87], 'LineWidth', 0.9);
434: ylabel(h.axHyd, 'Q (m^3/s)');
435: xlabel(h.axHyd, 'Time (min)');
436: title(h.axHyd, 'Hyetograph & hydrograph (live)', 'Color', COL.inkDim);
437:
438: barPanel = uipanel(chartGrid, 'BackgroundColor', COL.panel, 'BorderType', 'none');
439: bg = uigridlayout(barPanel, [1 1]);
440: bg.Padding = [8 6 8 6];
441: bg.BackgroundColor = COL.panel;
442:
443: h.axBar = uiaxes(bg);
444: styleAxes(h.axBar, COL);
445: h.axBar.Color = COL.panel;
446: h.bar = bar(h.axBar, [1 2], [0 0], 0.55, 'FaceColor', 'flat');
447: h.axBar.XLim = [0.4 2.6];
448: h.axBar.XTick = [1 2];
449: h.axBar.XTickLabel = {'Required Q', 'Capacity'};
450: ylabel(h.axBar, 'Q (m^3/s)');
451: title(h.axBar, 'Capacity vs peak demand', 'Color', COL.inkDim);
452: hold(h.axBar, 'on');
453: h.barTxt1 = text(h.axBar, 1, 0, '', 'Color', COL.ink, 'FontName', MONO, ...
454:   'FontSize', 10, 'HorizontalAlignment', 'center', 'VerticalAlignment', 'bottom');
455: h.barTxt2 = text(h.axBar, 2, 0, '', 'Color', COL.ink, 'FontName', MONO, ...
456:   'FontSize', 10, 'HorizontalAlignment', 'center', 'VerticalAlignment', 'bottom');
457:
458: %% =====
459: % START
460: % =====

```

```

461: recompute();
462:
463: tmr = timer('ExecutionMode', 'fixedRate', 'Period', 0.05, ...
464:             'BusyMode', 'drop', 'TimerFcn', @onTick);
465: fig.CloseRequestFcn = @onClose;
466: start(tmr);
467:
468: %% =====
469: % NESTED FUNCTIONS
470: % =====
471:
472: function setField(name, value)
473:     st.(name) = value;
474:     if strcmp(name, 'override')
475:         if value, h.efManual.Enable = 'on'; else, h.efManual.Enable = 'off'; end
476:     end
477:     if strcmp(name, 'useTc')
478:         if value, h.ddDur.Enable = 'off'; else, h.ddDur.Enable = 'on'; end
479:     end
480:     recompute();
481: end
482:
483: function recompute()
484:     % ---- time of concentration -----
485:     st.tc = time_of_concentration(st.L, st.Sc);
486:
487:     % The Rational Method assumes the storm lasts exactly as long as
488:     % the time of concentration, because that is when the whole
489:     % catchment is contributing at once and the peak flow occurs.
490:     if st.useTc
491:         st.durUse = st.tc;
492:     else
493:         st.durUse = st.durMin;
494:     end
495:
496:     % ---- hydrology -----
497:     if st.override
498:         st.Ipeak = st.manualI;
499:         st.exact = false;
500:     else
501:         [st.Ipeak, st.exact] = idf_intensity(st.durUse, st.Tr);
502:     end
503:     st.Qreq = rational_Q(st.C, st.Ipeak, st.A);
504:
505:     % ---- hydraulics -----
506:     [st.Qfull, st.Vfull] = capacityFast();
507:     st.yQmax = YMAX_RATIO * st.D;
508:     st.Qmax = manning_Q(st.yQmax, st.D, st.S, st.n);
509:     [st.yflow, st.Vflow] = solveDepth(st.Qreq);
510:
511:     [st.status,
... (remaining code in this segment omitted in preview)

```

Recompute function: complete calculation pipeline

Approximate code line range: 651 - 780

```

651:         st.rain.len(end+1) = 0.02 + 0.025*rand();
652:         st.rain.v(end+1) = 0.030 + 0.020*rand();
653:     end
654:     if ~isempty(st.rain.x)
655:         st.rain.y = st.rain.y - st.rain.v;
656:         st.rain.x = st.rain.x - 0.004;
657:         keep = st.rain.y > 0.55;
658:         st.rain.x = st.rain.x(keep);
659:         st.rain.y = st.rain.y(keep);
660:         st.rain.len = st.rain.len(keep);
661:         st.rain.v = st.rain.v(keep);
662:     end

```

```

663:         nd = numel(st.rain.x);
664:         if nd > 0
665:             rx3 = [st.rain.x; st.rain.x - 0.006; NaN(1, nd)];
666:             ry3 = [st.rain.y; st.rain.y - st.rain.len; NaN(1, nd)];
667:             h.rain.XData = rx3(:);
668:             h.rain.YData = ry3(:);
669:         else
670:             h.rain.XData = NaN; h.rain.YData = NaN;
671:         end
672:
673:         ta = linspace(0, 1, 20);
674:         ax0 = 0.26; ay0 = 0.55;
675:         ax1 = pcx - rx*0.8; ay1 = pcy + ry*0.4;
676:         axm = (ax0+ax1)/2; aym = min(ay0, ay1) - 0.06;
677:         h.arrow.XData = (1-ta).^2*ax0 + 2*(1-ta).*ta*axm + ta.^2*ax1;
678:         h.arrow.YData = (1-ta).^2*ay0 + 2*(1-ta).*ta*aym + ta.^2*ay1;
679:
680:         th = linspace(0, 2*pi, 180);
681:         h.pipe.XData = pcx + rx*cos(th);
682:         h.pipe.YData = pcy + ry*sin(th);
683:         h.crown.XData = [pcx-rx pcx+rx];
684:         h.crown.YData = [pcy+ry pcy+ry];
685:
686:         % Real hydraulics: the depth is solved from Manning's equation by
687:         % bisection every frame, not scaled proportionally.
688:         y_t = solveDepth(Q_t);
689:         yf = y_t / st.D;
690:
691:         if yf <= 0
692:             h.water.XData = NaN; h.water.YData = NaN;
693:             h.waterLine.XData = NaN; h.waterLine.YData = NaN;
694:         else
695:             phi = acos(max(-1, min(1, 1 - 2*yf)));
696:             ph = linspace(-phi, phi, 120);
697:             wx = pcx + rx*sin(ph);
698:             wy = pcy - ry*cos(ph);
699:             % The arc runs from one end of the water surface, round the
700:             % invert, back to the other end. patch closes the polygon
701:             % along the surface chord - exactly the circular segment.
702:             h.water.XData = wx;
703:             h.water.YData = wy;
704:
705:             surfY = pcy - ry*cos(phi);
706:             xs = linspace(pcx - rx*sin(phi), pcx + rx*sin(phi), 40);
707:             h.waterLine.XData = xs;
708:             h.waterLine.YData = surfY + 0.004*sin(12*xs + 8*st.simT*2*pi);
709:         end
710:
711:         if yf >= 0.999
712:             h.water.FaceColor = COL.danger;
713:         elseif yf >= 0.75
714:             h.water.FaceColor = COL.warn;
715:         else
716:             h.water.FaceColor = COL.water;
717:         end
718:
719:         h.txtPipe.Position(1) = pcx;
720:         h.txtPipe.Position(2) = pcy - ry - 0.06;
721:         h.txtPipe.String = sprintf('...
722:             'pipe D %.2f m - y/D = %.0f %% - Q = %.4f m^3/s', ...
723:             st.D, 100*yf, Q_t);
724:         h.txtClock.String = sprintf('%.1f / %.1f min', tMin, st.durUse);
725:     end
726:
727:     function drawHydro()
728:         if numel(st.hist.t) < 2, return; end
729:         t = st.hist.t; ii = st.hist.i; qq = st.hist.q;
730:
731:         maxI = max(st.Ipeak*1.15, 1e-6);
732:         maxQ = max([st.Qfull*1.15, max(qq)*1.15, 1e-6]);
733:
734:         yyax

```

... (remaining code in this segment omitted in preview)

Time of concentration calculation (Kirpich)

Approximate code line range: 781 - 820

```
781:         fprintf('=====\n\n');
782:
783:         fprintf('--- 1. DESIGN INPUTS -----\n');
784:         fprintf(' Catchment area      A = %8.3f ha\n', A);
785:         fprintf(' Runoff coefficient    C = %8.3f\n', C);
786:         fprintf(' Overland flow length  L = %8.0f m\n', st.L);
787:         fprintf(' Catchment slope      Sc = %8.4f m/m\n', st.Sc);
788:         fprintf(' Return period        Tr = %8.0f yr\n', st.Tr);
789:         fprintf(' Pipe diameter        D = %8.3f m\n', D);
790:         fprintf(' Pipe slope           S = %8.4f m/m\n', S);
791:         fprintf(' Manning roughness    n = %8.4f\n\n', n);
792:
793:         fprintf('--- 2. TIME OF CONCENTRATION -----\n');
794:         fprintf(' Kirpich: tc = 0.0195 * L^0.77 * Sc^-0.385 [min, m, m/m]\n');
795:         fprintf(' tc = 0.0195 * %.0f^0.77 * %.4f^-0.385 = %.2f min\n', ...
796:             st.L, st.Sc, st.tc);
797:         if st.useTc
798:             fprintf(' Storm duration set equal to tc = %.2f min\n\n', st.durUse);
799:         else
800:             fprintf(' Storm duration selected manually = %.0f min\n', st.durUse);
801:             fprintf(' NOTE: the Rational Method assumes duration = tc.\n\n');
802:         end
803:
804:         fprintf('--- 3. HYDROLOGY -----\n');
805:         if st.override
806:             fprintf(' Intensity (manual)    i = %8.2f mm/hr\n', st.Ipeak);
807:         else
808:             fprintf(' Khulna IDF intensity i = %8.2f mm/hr', st.Ipeak);
809:             if st.exact
810:                 fprintf(' (exact table entry)\n');
811:             else
812:                 fprintf(' (interpolated)\n');
813:             end
814:         end
815:         fprintf(' Rational Method      Q = C*i*A/360\n');
816:         fprintf('                      = %.3f * %.2f * %.3f / 360\n', C, st.Ipeak, A);
817:         fprintf('                      = %8.4f m^3/s (required)\n\n', st.Qreq);
818:
819:         [~, ~, Dtab, Ttab, Itab] = idf_intensity(60, 10);
820:         fprintf(' Stored Khulna IDF table, intensity in mm/hr:\n');
```

IDF rainfall intensity lookup and interpolation

Approximate code line range: 821 - 1010

```
821:         fprintf(' duration');
822:         fprintf('%9s', 'T=2', 'T=5', 'T=10', 'T=20', 'T=30', 'T=50', 'T=100');
823:         fprintf('\n');
824:         for r = 1:numel(Dtab)
825:             fprintf(' %7.0f min', Dtab(r));
826:             fprintf('%9.2f', Itab(r,:));
827:             fprintf('\n');
828:         end
829:         fprintf('\n');
830:
831:         fprintf('--- 4. HYDRAULICS -----\n');
832:         [~, ~, QmaxGrid, yQmaxGrid] = manning_capacity(D, S, n);
833:         [yB, VB, surch, itB, eaB] = normal_depth(st.Qreq, D, S, n);
834:
835:         fprintf(' Full-pipe capacity Qfull = %8.4f m^3/s\n', st.Qfull);
836:         fprintf(' Full-pipe velocity Vfull = %8.4f m/s\n', st.Vfull);
```

```

837:         fprintf(' Maximum discharge   Qmax = %8.4f m^3/s at y/D = %.4f\n', ...
838:                 QmaxGrid, yQmaxGrid/D);
839:         fprintf(' Theory: peak at y/D = 0.9382, Qmax/Qfull = 1.0757\n');
840:         fprintf(' Computed ratio           = %.4f\n\n', QmaxGrid/st.Qfull);
841:
842:         fprintf(' Bisection for the flow depth:\n');
843:         fprintf('   iterations           = %d\n',         itB);
844:         fprintf('   relative error ea      = %.3e %%\n',   eaB);
845:         fprintf('   flow depth           y       = %8.4f m   (y/D = %.1f %%) \n', ...
846:                 yB, 100*yB/D);
847:         fprintf('   flow velocity    V       = %8.4f m/s\n', VB);
848:         if surch
849:             fprintf(' PIPE IS SURCHARGED - demand exceeds Qmax\n');
850:         end
851:         fprintf('\n VERDICT: %s\n', st.status);
852:         fprintf(' Capacity / demand ratio = %.3f\n\n', st.Qfull/max(st.Qreq, eps));
853:
854:         fprintf('--- 5. HAND-CALCULATION CHECK -----\n');
855:         Afull_hand = pi*D^2/4;
856:         Rfull_hand = D/4;
857:         Qfull_hand = (1/n)*Afull_hand*Rfull_hand^(2/3)*sqrt(S);
858:         Qreq_hand  = C*st.Ipeak*A/360;
859:         Qcheck     = manning_Q(yB, D, S, n);
860:         fprintf('   A_full = pi*D^2/4                = %10.6f m^2\n', Afull_hand);
861:         fprintf('   R_full = D/4                    = %10.6f m\n',   Rfull_hand);
862:         fprintf('   Q_full by hand                 = %10.6f m^3/s\n', Qfull_hand);
863:         fprintf('   Q_full from the code          = %10.6f m^3/s\n', st.Qfull);
864:         fprintf('   difference                    = %10.3e\n\n', abs(st.Qfull-Qfull_hand));
865:         fprintf('   Q_req by hand                 = %10.6f m^3/s\n', Qreq_hand);
866:         fprintf('   Q_req from the code          = %10.6f m^3/s\n', st.Qreq);
867:         fprintf('   difference                    = %10.3e\n\n', abs(st.Qreq-Qreq_hand));
868:         fprintf('   Manning(y_solved) back-subbed = %10.6f m^3/s\n', Qcheck);
869:         fprintf('   target Q_req                 = %10.6f m^3/s\n', st.Qreq);
870:         fprintf('   residual                     = %10.3e m^3/s\n\n', abs(Qcheck-st.Qreq));
871:
872:         fprintf('--- 6. RUNOFF VOLUME -----\n');
873:         nSeg = 200;
874:         tv = linspace(0, st.durUse, nSeg+1); % minutes
875:         iv = triangular_hyetograph(tv, st.durUse, st.Ipeak);
876:         qv = rational_Q(C, iv, A);
877:         ts = tv * 60; % minutes -> seconds
878:
879:         Vol_trapz = trapz(ts, qv);
880:         Vol_simpson = simpson13(ts, qv);
881:         % Exact: the triangle encl
... (remaining code in this segment omitted in preview)

```

Rational Method discharge calculation

Approximate code line range: 1011 - 1055

```

1011: % LIMITATION, to be stated in the report: short-duration rainfall gauges
1012: % are scarce in Bangladesh, so durations below about 3 hours are
1013: % DISAGGREGATED from daily records rather than directly measured.
1014: %
1015: % INTERPOLATION - two passes of interp1, each in the variable that makes
1016: % the relationship straight, which is far more accurate than interpolating
1017: % the raw numbers:
1018: %
1019: % pass 1, along DURATION: IDF curves are close to straight lines on
1020: % log-log axes, so we interpolate log(i) against log(t) and take the
1021: % exponential. Interpolating i against t linearly across a 15 min to
1022: % 1440 min span would cut the corner off a strongly convex curve.
1023: %
1024: % pass 2, across RETURN PERIOD: for a Gumbel distribution the depth is
1025: % EXACTLY linear in the reduced variate
1026: % y = -ln(-ln(1 - 1/T)),
1027: % so interpolating against y rather than against T is not an
1028: % approximation at all - it reproduces the fitted distribution.

```

```

1029: %
1030: % Querying any of the 70 tabulated points returns the table value exactly.
1031:
1032: if nargin < 2
1033:     error('idf_intensity: duration and return period are both required');
1034: end
1035:
1036: % ----- stored table -----
1037: Dtab = [15; 30; 60; 90; 120; 180; 300; 360; 720; 1440]; % minutes
1038: Ttab = [2, 5, 10, 20, 30, 50, 100]; % years
1039:
1040: % Intensity in mm/hr. Rows match Dtab, columns match Ttab.
1041: % Reading the shape: rightwards the numbers grow (rarer storms are more
1042: % intense); downwards they shrink (longer storms are gentler per hour).
1043: Itab = [ 89.60 165.16 218.15 270.24 300.60 338.87 390.91
1044:         58.14 103.98 135.76 166.84 184.91 207.65 238.51
1045:         37.73 65.46 84.48 103.01 113.75 127.24 145.53
1046:         29.30 49.94 64.01 77.69 85.60 95.54 109.00
1047:         24.49 41.21 52.58 63.59 69.97 77.97 88.79
1048:         19.01 31.44 39.84 47.96 52.66 58.54 66.51
1049:         13.83 22.35 28.09 33.62 36.81 40.81 46.21
1050:         12.34 19.79 24.79 29.61 32.39 35.87 40.58
1051:         8.01 12.46 15.43 18.28 19.93 21.98 24.76
1052:         5.20 7.84 9.60 11.29 12.26 13.47 15.11 ];
1053:
1054: % ----- clamp the query to the table range -----
1055: % Extrapolating a rainfall table means inventing data, so clamp and warn.

```

Circular pipe geometry calculation

Approximate code line range: 1056 - 1145

```

1056: if durMin < min(Dtab) || durMin > max(Dtab)
1057:     warning('idf_intensity:duration', ...
1058:         'Duration %.4g min is outside the table (15-1440 min). Clamped.', durMin);
1059:     durMin = min(max(durMin, min(Dtab)), max(Dtab));
1060: end
1061: if Tr < min(Ttab) || Tr > max(Ttab)
1062:     warning('idf_intensity:returnPeriod', ...
1063:         'Return period %.4g yr is outside the table (2-100 yr). Clamped.', Tr);
1064:     Tr = min(max(Tr, min(Ttab)), max(Ttab));
1065: end
1066:
1067: % ----- pass 1: log-log interpolation along duration -----
1068: nT = numel(Ttab);
1069: iAtDur = zeros(1, nT);
1070: for k = 1:nT
1071:     iAtDur(k) = exp( interp1(log(Dtab), log(Itab(:,k)), log(durMin), 'linear') );
1072: end
1073:
1074: % ----- pass 2: interpolation in the Gumbel reduced variate ---
1075: yTab = -log(-log(1 - 1./Ttab)); % reduced variate of the table
1076: yQ = -log(-log(1 - 1/Tr)); % reduced variate of the query
1077:
1078: i = interp1(yTab, iAtDur, yQ, 'linear');
1079:
1080: % ----- did we land on a table entry? -----
1081: isExact = any(abs(Dtab - durMin) < 1e-9) && any(abs(Ttab - Tr) < 1e-9);
1082: end
1083:
1084:
1085: function tc = time_of_concentration(L, Sc)
1086: %TIME_OF_CONCENTRATION Kirpich formula.
1087: %
1088: % tc = time_of_concentration(L, Sc)
1089: %
1090: % input:
1091: % L = length of the longest overland flow path (m)
1092: % Sc = average slope along that path (m/m)

```

```

1093: % output:
1094: %   tc = time of concentration (minutes)
1095: %
1096: %       tc = 0.0195 * L^0.77 * Sc^(-0.385)
1097: %
1098: % WHY THIS MATTERS
1099: %   The Rational Method assumes the storm lasts exactly as long as the
1100: %   time of concentration - the time water needs to travel from the
1101: %   furthest corner of the catchment to the inlet. Only then is the whole
1102: %   catchment contributing at once, which is when the peak flow occurs.
1103: %   A shorter storm never engages the whole area; a longer one is less
1104: %   intense. So tc is what tells you WHICH row of the IDF table to read.
1105: %
1106: %   Getting this wrong is the most common misuse of the Rational Method.
1107: %   Reading a 24-hour intensity for a catchment whose tc is 20 minutes
1108: %   under-estimates the design flow by an order of magnitude.
1109: %
1110: %   Kirpich was calibrated on small rural catchments; for paved urban
1111: %   surfaces it tends to give short values, which is conservative here
1112: %   because shorter duration means higher intensity.
1113: %
1114: % if nargin < 2
1115: %     error('time_of_concentration: two input arguments required (L, Sc)');
1116: % end
1117: % if L <= 0
1118: %     error('time_of_concentration: flow length L must be positive');
1119: % end
1120: % if Sc <= 0
1121: %     error('time_of_concentration: catchment slope Sc must be positive');
1122: % end
1123: %
1124: % tc = 0.0195 * L^0.77 * Sc^(-0.385);
1125: % end
1126: %
1127: %
1128: function Q = rational_Q(C, i, A)
1129: %RATIONAL_Q Peak runoff discharge by the Rational Method.
1130: %
1131: % Q = rational_Q(C, i, A)
1132: %
1133: % input:
1134: %   C = runoff coefficient (0 - 1)
1135: %   i = rainfall intensity (mm/hr)
1136: %   A = catchment area (hectares)
1137: % output:
1138: %   Q = peak discharge (m^3/s)
1139: %
1140: % The Rational Method in raw form is Q = C*i*A. The 360 is a UNIT
1141: % CONVERSION, not an empirical constant:
1142: %
1143: %   i [mm/hr] * A [ha] = (i/1000
... (remaining code in this segment omitted in preview)

```

Manning equation for partial flow

Approximate code line range: 1146 - 1215

```

1146: %
1147: % Vectorised, so one call converts a whole hyetograph into a hydrograph.
1148: %
1149: % if nargin < 3
1150: %     error('rational_Q: three input arguments required (C, i, A)');
1151: % end
1152: %
1153: % Q = C .* i .* A / 360;
1154: % end
1155: %
1156: %
1157: function [Aflow, P, R, Tw, theta] = pipe_geometry(y, D)

```

```

1158: %PIPE_GEOMETRY Flow geometry of a part-full circular pipe.
1159: %
1160: % [Aflow, P, R, Tw, theta] = pipe_geometry(y, D)
1161: %
1162: % input:
1163: % y = depth of flow from the pipe invert (m)
1164: % D = internal pipe diameter (m)
1165: % output:
1166: % Aflow = wetted cross-sectional flow area (m^2)
1167: % P = wetted perimeter (m) - the pipe wall the water touches, NOT
1168: % including the free surface, because air causes no friction
1169: % R = hydraulic radius = Aflow / P (m)
1170: % Tw = top width of the water surface (m)
1171: % theta = wetted central angle (rad)
1172: %
1173: % cos(theta/2) = 1 - 2*y/D -> theta = 2*acos(1 - 2*y/D)
1174: %
1175: % Aflow = (D^2/8) * (theta - sin(theta)) circular segment area
1176: % P = D*theta/2 arc length
1177: % R = Aflow / P
1178: % Tw = D*sin(theta/2)
1179: %
1180: % CHECK at y = D (pipe full): theta = 2*pi, so Aflow = pi*D^2/4,
1181: % P = pi*D and R = D/4 - the familiar full-pipe values.
1182: %
1183: if nargin < 2
1184: error('pipe_geometry: two input arguments required (y, D)');
1185: end
1186: if D <= 0
1187: error('pipe_geometry: diameter D must be positive');
1188: end
1189: %
1190: y = min(max(y, 0), D);
1191: %
1192: theta = 2 * acos(1 - 2*y/D);
1193: %
1194: Aflow = (D^2/8) * (theta - sin(theta));
1195: P = D * theta / 2;
1196: Tw = D * sin(theta/2);
1197: %
1198: if P > 0
1199: R = Aflow / P;
1200: else
1201: R = 0;
1202: end
1203: end
1204: %
1205: %
1206: function [Q, V, R, Aflow] = manning_Q(y, D, S, n)
1207: %MANNING_Q Discharge of a part-full circular pipe by Manning's equation.
1208: %
1209: % [Q, V, R, Aflow] = manning_Q(y, D, S, n)
1210: %
1211: % Q = (1/n) * A * R^(2/3) * S^(1/2)
1212: %
1213: % Flow rises with area, rises with hydraulic radius, rises with the square
1214: % root of slope, and falls with roughness. The exponents are empirical -
1215: % Manning fitted them to channel data in the 1890s.

```

Pipe capacity and maximum discharge search

Approximate code line range: 1216 - 1285

```

1216: %
1217: % SI UNITS ONLY. The 1/n term carries hidden units; US customary form
1218: % needs an extra factor of 1.49.
1219: %
1220: % Q(y) is NOT monotonic over 0 <= y <= D. Near the crown the wall curves
1221: % inward and adds wetted perimeter faster than flow area, so R falls and

```

```

1222: % Q peaks at about y/D = 0.938 before dropping back to the full-pipe
1223: % value. This is why normal_depth brackets on [0, 0.938D].
1224:
1225: if nargin < 4
1226:     error('manning_Q: four input arguments required (y, D, S, n)');
1227: end
1228: if S <= 0
1229:     error('manning_Q: slope S must be positive');
1230: end
1231: if n <= 0
1232:     error('manning_Q: roughness n must be positive');
1233: end
1234:
1235: [Aflow, ~, R] = pipe_geometry(y, D);
1236:
1237: Q = (1/n) * Aflow * R^(2/3) * sqrt(S);
1238:
1239: if Aflow > 0
1240:     V = Q / Aflow;
1241: else
1242:     V = 0;
1243: end
1244: end
1245:
1246:
1247: function [Qfull, Vfull, Qmax, yQmax] = manning_capacity(D, S, n)
1248: %MANNING_CAPACITY Full-pipe capacity and true maximum discharge.
1249: %
1250: % Qfull uses A = pi*D^2/4 and R = D/4 - the capacity quoted in design.
1251: %
1252: % Qmax is found by GRID SEARCH: manning_Q is evaluated at 2001 trial
1253: % depths and the largest picked with max. That is the "scan and take the
1254: % best" idea from the Fundamentals lecture. Theory says yQmax/D = 0.9382
1255: % with Qmax/Qfull = 1.0757, so the grid result independently checks the
1256: % theory - and vice versa.
1257:
1258: if nargin < 3
1259:     error('manning_capacity: three input arguments required (D, S, n)');
1260: end
1261:
1262: Afull = pi * D^2 / 4;
1263: Rfull = D / 4;
1264:
1265: Qfull = (1/n) * Afull * Rfull^(2/3) * sqrt(S);
1266: Vfull = Qfull / Afull;
1267:
1268: yGrid = linspace(1e-6, D, 2001);
1269: qGrid = zeros(size(yGrid));
1270: for k = 1:numel(yGrid)
1271:     qGrid(k) = manning_Q(yGrid(k), D, S, n);
1272: end
1273:
1274: [Qmax, kBest] = max(qGrid);
1275: yQmax = yGrid(kBest);
1276: end
1277:
1278:
1279: function [root, fx, ea, iter] = bisection_root(func, xl, xu, es, maxit)
1280: %BISECTION_ROOT Root of a function by the bisection method.
1281: %
1282: % [root, fx, ea, iter] = bisection_root(func, xl, xu, es, maxit)
1283: %
1284: % input:
1285: % func = handle to the function being solved, f(x) = 0

```

Bisection root solver

Approximate code line range: 1286 - 1365

```

1286: % xl,xu = lower and upper guesses that bracket the root
1287: % es = desired relative error in % (default 0.0001)
1288: % maxit = maximum allowable iterations (default 50)
1289: %
1290: % This is bisection.m from the "Finding Roots of Equations" lecture, renamed
1291: % so it cannot collide with a copy already in your own CE 206 folder.
1292: % Halve the interval, keep whichever half still shows a sign change. The
1293: % error falls by a factor of 2 every pass, so 23 iterations shrink it by
1294: % roughly 8 million.
1295: %
1296: % The test func(xl)*func(xr) < 0 uses multiplication to compare signs: a
1297: % negative product means the two values straddle zero.
1298: %
1299: if nargin < 3
1300:     error('bisection_root: at least 3 input arguments required');
1301: end
1302: %
1303: test = func(xl) * func(xu);
1304: if test > 0
1305:     error('bisection_root: no sign change over [xl, xu], root is not bracketed');
1306: end
1307: %
1308: if nargin < 4 || isempty(es), es = 0.0001; end
1309: if nargin < 5 || isempty(maxit), maxit = 50; end
1310: %
1311: iter = 0;
1312: xr = xl;
1313: ea = 100;
1314: %
1315: while (1)
1316:     xrold = xr;
1317:     xr = (xl + xu) / 2;
1318:     iter = iter + 1;
1319: %
1320:     if xr == 0
1321:         ea = abs((xr - xrold)/xr) * 100;
1322:     end
1323: %
1324:     test = func(xl) * func(xr);
1325:     if test < 0
1326:         xu = xr;
1327:     elseif test > 0
1328:         xl = xr;
1329:     else
1330:         ea = 0;
1331:     end
1332: %
1333:     if ea <= es || iter >= maxit
1334:         break
1335:     end
1336: end
1337: %
1338: root = xr;
1339: fx = func(xr);
1340: end
1341: %
1342: %
1343: function [y, V, surcharged, iter, ea] = normal_depth(Qreq, D, S, n, es, maxit)
1344: %NORMAL_DEPTH Depth of flow in a circular pipe carrying a given discharge.
1345: %
1346: % THE ROOT PROBLEM - the heart of the project.
1347: %
1348: % Manning's equation gives Q for a KNOWN depth. The design question is
1349: % the reverse: what depth carries a KNOWN Q? Substituting the circular
1350: % segment geometry into Manning's equation gives
1351: %
1352: % 
$$f(y) = (1/n) * A(y) * R(y)^{2/3} * \text{sqrt}(S) - Q_{\text{req}} = 0$$

1353: %
1354: % with y buried inside acos(1 - 2y/D), inside a subtraction, inside a
1355: % fractional power, inside a ratio. It cannot be rearranged for y. The
1356: % equation is transcendental and must be solved numerically.
1357: %

```

```

1358: % WHY BISECTION rather than Newton-Raphson:
1359: % 1. No derivative needed - differentiating that expression by hand is
1360: % genuinely unpleasant; bisection uses only function values.
1361: % 2. The bracket is guaranteed, so it cannot fail:
1362: %     y = 0      -> f = -Qreq      < 0
1363: %     y = yQmax  -> f = Qmax - Qreq > 0 (when the flow fits)
1364: % Newton-Raphson can diverge from a poor start. For a solver running
1365: % unattended 20 times a second inside a GUI, guaranteed convergence

```

Normal depth solution using bisection

Approximate code line range: 1366 - 1455

```

1366: %      beats fast convergence.
1367: %
1368: % WHY THE BRACKET STOPS AT 0.938D, NOT D:
1369: % Q(y) rises then falls, so over [0, D] there can be TWO depths giving
1370: % the same discharge. Bisection assumes ONE root in the bracket; give it
1371: % two and the answer is meaningless. Over [0, yQmax] the function is
1372: % strictly increasing, so the root is unique.
1373:
1374: if nargin < 4
1375:     error('normal_depth: four input arguments required (Qreq, D, S, n)');
1376: end
1377: if nargin < 5 || isempty(es), es = 0.0001; end
1378: if nargin < 6 || isempty(maxit), maxit = 50; end
1379:
1380: if Qreq <= 0
1381:     y = 0; V = 0; surcharged = false; iter = 0; ea = 0;
1382:     return
1383: end
1384:
1385: [~, ~, Qmax, yQmax] = manning_capacity(D, S, n);
1386:
1387: % No root exists - detect this BEFORE calling bisection rather than
1388: % handing it an unbracketed problem and hoping.
1389: if Qreq >= Qmax
1390:     y = D;
1391:     [~, V] = manning_Q(D, D, S, n);
1392:     surcharged = true;
1393:     iter = 0;
1394:     ea = NaN;
1395:     return
1396: end
1397:
1398: f = @(yy) manning_Q(yy, D, S, n) - Qreq; % anonymous function handle
1399:
1400: [y, ~, ea, iter] = bisect_root(f, 1e-9, yQmax, es, maxit);
1401:
1402: [~, V] = manning_Q(y, D, S, n);
1403: surcharged = false;
1404: end
1405:
1406:
1407: function i_t = triangular_hyetograph(t, Ddur, Ipeak)
1408: %TRIANGULAR_HYETOGRAPH Symmetric triangular design storm.
1409: %
1410: % i_t = triangular_hyetograph(t, Ddur, Ipeak)
1411: %
1412: % input:
1413: % t = time from the start of the storm, same units as Ddur
1414: % Ddur = total storm duration (minutes in this application)
1415: % Ipeak = peak rainfall intensity (mm/hr), from the IDF table
1416: % output:
1417: % i_t = rainfall intensity at time t (mm/hr), same size as t
1418: %
1419: % The IDF table gives ONE number: the average intensity over the whole
1420: % duration. Real rain builds, peaks and dies away, so a design storm
1421: % distributes that number into a time pattern:

```

```

1422: %
1423: %      i(t) = Ipeak * (t/tp)      for 0 <= t <= tp
1424: %      i(t) = Ipeak * (Ddur - t)/tp for tp < t <= Ddur
1425: %      tp = Ddur/2
1426: %
1427: % A real design would use an alternating-block or Huff distribution built
1428: % from local storm records; the triangle is chosen for transparency.
1429: %
1430: % Written with LOGICAL MASKS instead of a loop with an if inside,
1431: % following the vectorisation section of the Fundamentals lecture.
1432:
1433: if nargin < 3
1434:     error('triangular_hyetograph: three input arguments required');
1435: end
1436:
1437: tp = Ddur / 2;
1438: i_t = zeros(size(t));
1439:
1440: rising = (t >= 0) & (t <= tp);
1441: falling = (t > tp) & (t <= Ddur);
1442:
1443: i_t(rising) = Ipeak .* t(rising) / tp;
1444: i_t(falling) = Ipeak .* (Ddur - t(falling)) / tp;
1445:
1446: i_t = max(i_t, 0);
1447: end
1448:
1449:
1450: function I = simpson13(x, y)
1451: %SIMPSON13 Numerical integration by Simpson's 1/3 rule.
1452: %
1453: %      I = (h/3)*[ f(x0) + 4*sum(odd) + 2*sum(even) + f(xn) ]
1454: %
1455: % Simpson's rule fits a PARABOLA through each group of three consecutive

```

Storm hyetograph generation

Approximate code line range: 1456 - 1515

```

1456: % points, so it is exact for polynomials up to cubic order, whereas trapz
1457: % joins points with straight lines and is exact only for linear data. The
1458: % alternating 4s and 2s are the weights that fall out of integrating a
1459: % parabola through three equally spaced points.
1460: %
1461: % Requires an EVEN number of segments and EQUAL spacing. Both are checked,
1462: % because silently returning a wrong number is worse than an error.
1463:
1464: if nargin < 2
1465:     error('simpson13: two input arguments required (x, y)');
1466: end
1467:
1468: x = x(:);
1469: y = y(:);
1470:
1471: if numel(x) ~= numel(y)
1472:     error('simpson13: x and y must be the same length');
1473: end
1474:
1475: n = numel(x) - 1;
1476: if mod(n, 2) ~= 0
1477:     error('simpson13: needs an even number of segments (odd number of points)');
1478: end
1479:
1480: h = (x(end) - x(1)) / n;
1481:
1482: if max(abs(diff(x) - h)) > 1e-9 * max(1, abs(h))
1483:     error('simpson13: x must be equally spaced');
1484: end
1485:

```

```

1486: oddSum = sum(y(2:2:n));
1487: evenSum = sum(y(3:2:n));
1488:
1489: I = (h/3) * (y(1) + 4*oddSum + 2*evenSum + y(end));
1490: end
1491:
1492:
1493: function [status, level] = design_status(Qreq, Qcap, V, vmin, vmax)
1494: %DESIGN_STATUS Verdict on a trial storm sewer design.
1495: %
1496: % THE THREE CHECKS, in order of severity:
1497: % capacity Qcap < Qreq. The pipe cannot pass the flow. Water backs up,
1498: % manholes surcharge, the street floods. FAIL.
1499: % vmin Below about 0.6 m/s, grit and organic solids settle out
1500: % instead of being swept along, and the sewer silts up and
1501: % blocks. This is the SELF-CLEANSING velocity.
1502: % vmax Above about 3.0 m/s the suspended grit abrades the invert.
1503: %
1504: % Capacity is tested first because it governs: a pipe that cannot pass the
1505: % flow is broken regardless of its velocity.
1506: %
1507: % These criteria FIGHT each other. Enlarging a pipe to fix a capacity
1508: % failure makes the flow shallower and slower, pushing toward siltation;
1509: % steepening it raises velocity toward the scour limit and deepens the
1510: % excavation. Design is finding the window where all three hold at once.
1511:
1512: if nargin < 3
1513:     error('design_status: at least 3 input arguments required');
1514: end
1515: if nargin < 4 || isempty(vmin), vmin = 0.6; end

```

Simpson integration for runoff volume

Approximate code line range: 1516 - 1570

```

1516: if nargin < 5 || isempty(vmax), vmax = 3.0; end
1517:
1518: if Qcap < Qreq
1519:     status = 'UNDERSIZED PIPE - capacity below demand';
1520:     level = 3;
1521: elseif V < vmin
1522:     status = 'VELOCITY TOO LOW - siltation risk';
1523:     level = 2;
1524: elseif V > vmax
1525:     status = 'VELOCITY TOO HIGH - erosion risk';
1526:     level = 2;
1527: else
1528:     status = 'OK - design passes';
1529:     level = 1;
1530: end
1531: end
1532:
1533:
1534: function [i_T, x_T, u, alpha] = gumbel_fit(annualMax, durHr, Tr)
1535: %GUMBEL_FIT Gumbel (EV Type I) frequency analysis of annual maxima.
1536: %
1537: % [i_T, x_T, u, alpha] = gumbel_fit(annualMax, durHr, Tr)
1538: %
1539: % input:
1540: % annualMax = vector of annual maximum rainfall depths (mm) for one
1541: % duration, one value per year of record
1542: % durHr = the duration those depths refer to (hr)
1543: % Tr = return period(s) of interest (yr)
1544: % output:
1545: % i_T = rainfall intensity for each Tr (mm/hr)
1546: % x_T = rainfall depth for each Tr (mm)
1547: % u, alpha = fitted location and scale parameters (mm)
1548: %
1549: % THE IDEA

```

```

1550: % Take many years of records, keep the single largest storm from each
1551: % year, fit a distribution to that series of annual maxima, then use it
1552: % to answer "what depth has a 1-in-T chance each year?" The Gumbel
1553: % distribution is built for exactly this - it models the maximum of many
1554: % samples. This is the method behind the stored Khulna table.
1555: %
1556: % METHOD OF MOMENTS
1557: %     alpha = sqrt(6) * s / pi
1558: %     u     = mean - 0.5772 * alpha      (0.5772 = Euler's constant)
1559: %     x_T   = u - alpha * ln( -ln( 1 - 1/Tr ) )
1560: %
1561: % NOTE: this function is supplied so a real BMD annual maximum series can
1562: % be processed directly if your group obtains one. It is NOT used to
1563: % generate the stored table - that comes from the published source.
1564: %
1565: if nargin < 3
1566:     error('gumbel_fit: three input arguments required');
1567: end
1568: %
1569: x = annualMax(:);
1570: x = x(~isnan(x));

```

Design status PASS/WARN/FAIL logic

Approximate code line range: 1571 - 1625

```

1571: %
1572: N = numel(x);
1573: if N < 5
1574:     error('gumbel_fit: need at least 5 years of record, got %d', N);
1575: end
1576: %
1577: xbar = mean(x);
1578: s     = std(x); % n-1 degrees of freedom
1579: %
1580: alpha = sqrt(6) * s / pi;
1581: u     = xbar - 0.5772 * alpha;
1582: %
1583: Tr = Tr(:)';
1584: x_T = u - alpha * log(-log(1 - 1 ./ Tr));
1585: i_T = x_T / durHr;
1586: end
1587: %
1588: %
1589: function [u, alpha, resid] = gumbel_from_table()
1590: %GUMBEL_FROM_TABLE Recover the Gumbel parameters behind the stored table.
1591: %
1592: % [u, alpha, resid] = gumbel_from_table()
1593: %
1594: % output:
1595: % u     = location parameter for each duration row (mm)
1596: % alpha = scale parameter for each duration row (mm)
1597: % resid = largest absolute fitting residual per row (mm)
1598: %
1599: % For a Gumbel distribution the depth is exactly linear in the reduced
1600: % variate
1601: %     y = -ln(-ln(1 - 1/T)),
1602: % that is P = u + alpha*y.
1603: %
1604: % The source study fitted Gumbel to 62 years of BMD data. Least-squares
1605: % fitting a straight line to each row of the published table therefore
1606: % recovers the parameters that study obtained, and the residuals show how
1607: % cleanly the table follows the distribution. Small residuals confirm the
1608: % table really is a Gumbel product rather than a set of loose numbers -
1609: % which is a validation you can quote, using only published values and no
1610: % invented data.
1611: %
1612: % This is the Descriptive Statistics module applied in reverse: instead of
1613: % going from a sample to parameters, we go from published quantiles back

```

```

1614: % to the parameters that produced them.
1615:
1616: [~, ~, Dtab, Ttab, Itab] = idf_intensity(60, 10); % fetch the table
1617:
1618: y = -log(-log(1 - 1./Ttab)); % reduced variate, row vector
1619: nD = numel(Dtab);
1620:
1621: u = zeros(nD, 1);
1622: alpha = zeros(nD, 1);
1623: resid = zeros(nD, 1);
1624:
1625: for r = 1:nD

```

Gumbel EV-I fitting from annual maximum rainfall

Approximate code line range: 1626 - 1705

```

1626: P = Itab(r,:) .* (Dtab(r)/60); % intensity (mm/hr) -> depth (mm)
1627:
1628: % Least squares straight line, P = u + alpha*y, written out rather
1629: % than calling polyfit so the arithmetic is visible.
1630: n = numel(y);
1631: Sy = sum(y);
1632: SP = sum(P);
1633: Syy = sum(y.^2);
1634: SyP = sum(y .* P);
1635:
1636: alpha(r) = (n*SyP - Sy*SP) / (n*Syy - Sy^2);
1637: u(r) = (SP - alpha(r)*Sy) / n;
1638:
1639: resid(r) = max(abs(P - (u(r) + alpha(r)*y)));
1640: end
1641: end
1642: ``

```

Gumbel back-analysis from IDF table

Approximate code line range: 1706 - 1775

Overall Background Calculation Flow

1. Khulna IDF table provides rainfall intensity for duration and return period.
2. Kirpich equation calculates time of concentration.
3. Rational Method:
$$Q = C i A / 360$$

converts rainfall intensity into required runoff discharge.
4. Manning equation calculates pipe capacity:
$$Q = (1/n) A R^{2/3} S^{1/2}$$
5. Circular pipe geometry calculates area and hydraulic radius for partial flow.
6. Bisection solves the nonlinear normal-depth equation.
7. Velocity is checked against self-cleansing and erosion limits.
8. GUI displays the design status, animation, and graphs.